

위성궤도결정과 필터링 기말고사 보고서

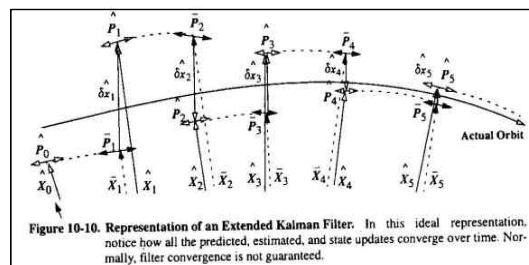
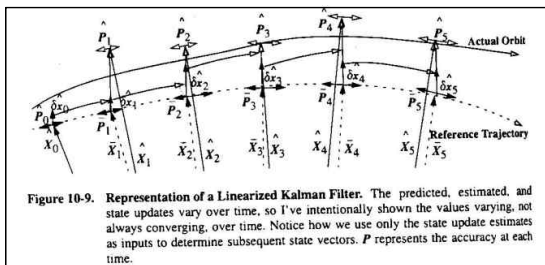
2022311224 이진진

1) Explain the POD process using EKF, and Draw a flowchart for the POD process.

Kalman Filter는 $\hat{X}_k$ (이전 시각의 state estimate), $\hat{P}_k$ (정확도 나타내는 Covariance Matrix), $z_{k+1}(obs)$ (현재 시각에 관측된 값), Q (Dynamics와의 Error), R_{k+1} (현재 시각 관측값의 error) 등의 조건이 주어지면 그 다음 시각의 $\hat{X}$ 와 $\hat{P}$ 를 알 수 있게 해주는 측정 기법이다. Kalman Filter의 장점은 계산이 빠르고 요구되는 메모리 용량이 적으며, 현재 시각에 실시간으로 State와 Covariance Matrix를 추정할 수 있다는 점이 있다. 또한, Batch Filter와 달리 시간이 변화하는 것을 고려하여 실시간으로 계산하므로 매 시간마다 변화하는 변수들을 반영할 수 있다. Kalman Filter의 종류에는 세 가지가 있으며 LKF(Linear Kalman Filter), EKF(Extended Kalman Filter), UKF(Unscented Kalman Filter)이다.

가장 먼저, LKF(Linear Kalman Filter)는 특정 State를 기준으로 하여 새로운 Reference trajectory를 만들어 내며 이를 바탕으로 propagation을 한다. Reference trajectory와 actual orbit 간의 차이를 계산하는 방식으로 위치를 추정하게 되며 첫 state부터 propagation을 이어 나가기 때문에 시간이 지남에 따라 오차가 누적될 수 있다. 따라서 최종 궤도를 계산하였을 때 actual orbit과의 차이가 많이 발생할 수 있다는 치명적인 단점이 있다. 이 단점을 보완하기 위해 사용하는 Filtering 방식이 EKF(Extended Kalman Filter)이다.

EKF에서는 관측된 지점마다 새로운 Trajectory를 생성하는 방식을 이용하여 오차가 누적되는 것을 방지할 수 있다. LKF에서의 $\hat{X}$ 와 $\hat{P}$ 는 $\delta x_{k+1}=0$ 을 기준으로 하지만 EKF에서는 현재 시각에 대해 update하며 매번 새로운 Reference Trajectory를 계산할 수 있다. 일련의 과정에 차이가 있음을 이해하기 위해 두 사진을 첨부하여 비교해보았다. (순서대로 LKF, EKF)



Algorithm: Extended Kalman Filter

Given $(\hat{X}_k, \hat{P}_k, Q, R_{k+1}, z_{k+1}(obs))$ 초기 조건을 전파하여

Want to know $\hat{X}_{k+1}, \hat{P}_{k+1}$ 현재의 $\hat{X}$ 와 $\hat{P}$ 를 계산하는 것이 EKF이다.

$$H = \frac{\partial g(X)}{\partial X} \Big|_{\bar{X}_{k+1}} \quad \text{observation model } y = g(X)$$

Partial Derivative Matrix는 H로 정의하며 위의 식과 같이 표현할 수 있다.

Prediction Prediction과 Update를 반복하여 매 시각 새로운 관측 값을 수정해나간다.

$$\text{PKEPLER}(\hat{X}_k, t_{k+1} - t_k \Rightarrow \bar{X}_{k+1}) \quad ; \text{Predicted State}$$

repeat each step
 → get new reference trajectory

동역학 모델을 이용하여 기존의 추정 값을 전파하여 계산된 state를 얻는다.

위 과정으로 새로운 reference trajectory를 얻을 수 있다.

$$F(t) = \frac{\partial \hat{X}_{k+1}}{\partial \hat{X}_{k+1}} \approx \frac{\partial \bar{X}_{k+1}}{\partial \bar{X}_{k+1}} \quad \begin{array}{l} \text{Because we can't know estimated state} \\ \text{추정된 state를 알 수 없기 때문에} \\ \text{계산한 state를 통해 근사한다.} \end{array}$$

$\Phi(t_{k+1}, t_k) = F(t)\Phi(t_{k+1}, t_k)$ 다음 식을 통해 State Transition Matrix를 계산할 수 있다.

→ Using numerical integration get $\Phi(t_{k+1}, t_k)$

$$\begin{array}{ll} \delta \bar{X}_{k+1} = 0 & ; \text{predicted state update} \\ \bar{P}_{k+1} = \Phi \hat{P}_k \Phi^T + Q & ; \text{predicted error covariance} \end{array} \quad \begin{array}{l} \text{계산된 Matrix로 Predicted state와} \\ \text{Covariance Matrix를 구할 수 있다.} \end{array}$$

Update

$$\tilde{b}_{k+1} = z_{k+1} - H_{k+1} \bar{X}_{k+1} \quad \text{Kalman 식에 의해 residual을 다음과 같이 정의한다.}$$

$$K_{k+1} = \bar{P}_{k+1} H_{k+1}^T [H_{k+1} \bar{P}_{k+1} H_{k+1}^T + R_{k+1}]^{-1} \quad ; \text{Kalman Gain}$$

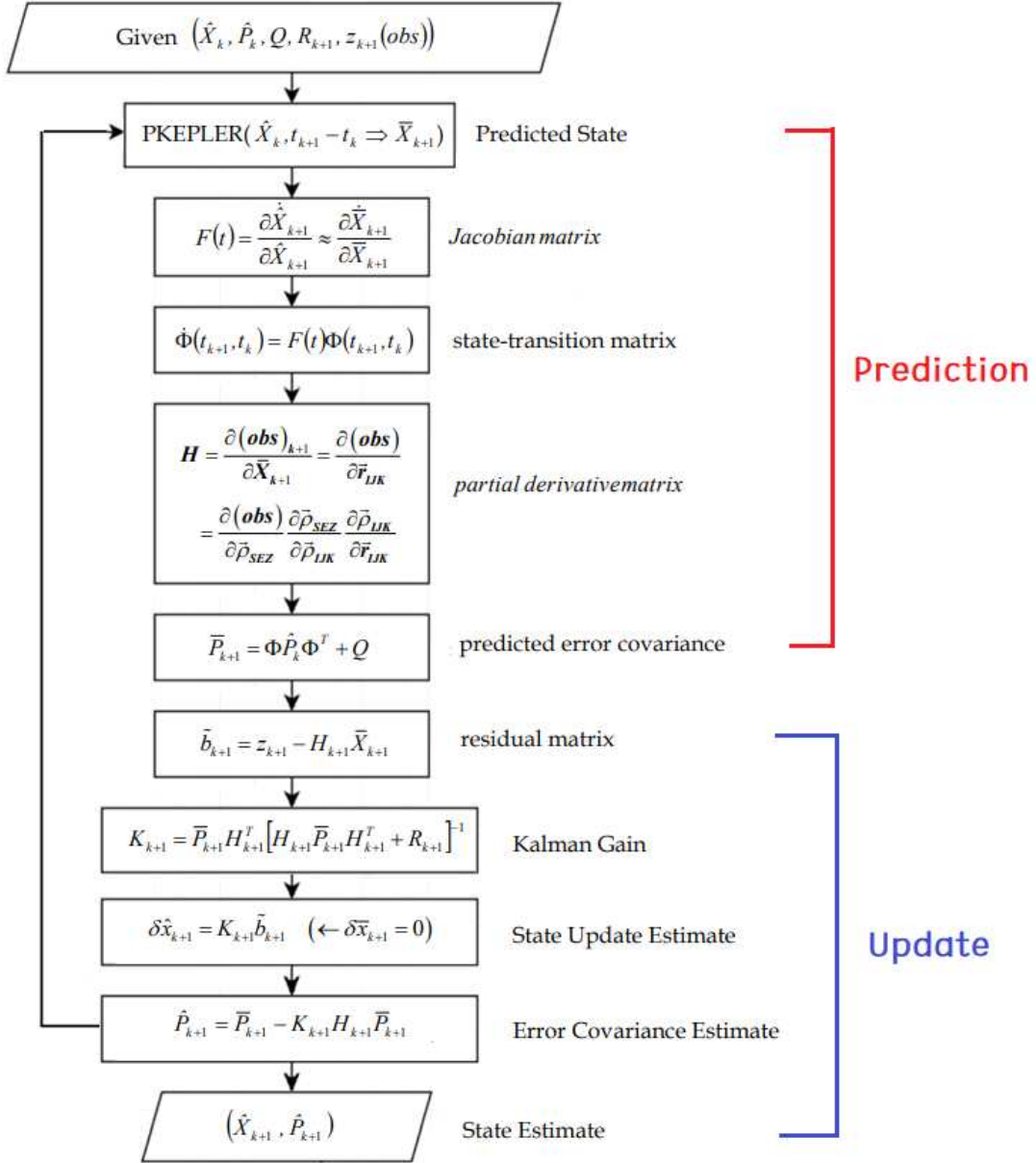
$$\delta \hat{X}_{k+1} = K_{k+1} \tilde{b}_{k+1} \quad (\leftarrow \delta \bar{X}_{k+1} = 0) \quad ; \text{State Update Estimate}$$

$$\hat{P}_{k+1} = \bar{P}_{k+1} - K_{k+1} H_{k+1} \bar{P}_{k+1} \quad ; \text{Error Covariance Estimate}$$

$$\hat{X}_{k+1} = \bar{X}_{k+1} + \delta \hat{X}_{k+1} \quad ; \text{State Estimate}$$

$$\begin{aligned} \delta \hat{x}(0|k+n) &= (A_{new}^T W_{new} A_{new} + \hat{P}_k^{-1})^{-1} (A_{new}^T W_{new} \tilde{b}_{new} + A_k^T W_k \tilde{b}_k) \\ \hat{P}(0|k+n) &= (A_{new}^T W_{new} A_{new} + \hat{P}_k^{-1})^{-1} \end{aligned}$$

위의 식을 변형하여 Kalman gain, Error Covariance, State Estimate를 순차적으로 계산할 수 있다.



2) Explain how to build up measurement model, and Draw a flowchart for the measurement model.

POD(Precise orbit determination) 과정이란 궤도를 cm 단위까지 매우 정밀하게 결정하는 것을 말하며 계산된 관측 값과 실제 관측 값의 차이(residual)를 줄이는 것이 최종목표이다. 그 중 Non-linear least square을 통한 differential correction을 사용하는 것이 이를 가장 효과적으로 수행할 수 있는 방법이다. 이 방법은 Nominal state의 초기 값(y_{n0})을 추정하고 동일한 시간에 대해 반복적으로 differential을 거쳐 오차를 최소화하는 것이다.

A sensor can measure **range(ρ)**, **azimuth(β)** and **elevation(el)** at each time t_i

$$y_{0i} = \begin{bmatrix} \rho_0 \\ \beta_0 \\ el_0 \end{bmatrix} \text{ at } t_i \quad \text{ti 시간에서의 Range, Azimuth, Elevation을 설정}$$

We need to compute predicted values

$$y_{ci} = \begin{bmatrix} \rho_c \\ \beta_c \\ el_c \end{bmatrix} \text{ at } t_i \quad y_{ci}; a \text{ non-linear function of } \vec{r}, \vec{v}$$

초기시간 t_0 에 대한 dynamic model을 통해 초기 State X를 나타낼 수 있다.

state $X = (\vec{r}(t_0), \vec{v}(t_0))$, at initial time $\vec{r}(t_0), \vec{v}(t_0)$ are given
dynamic model $f(\vec{r}(t_0), \vec{v}(t_0))$

$$\Rightarrow \vec{r}(t_i), \vec{v}(t_i) \Rightarrow \text{measurement model, } y(\vec{r}(t_i), \vec{v}(t_i)) \Rightarrow \begin{bmatrix} \rho_c \\ \beta_c \\ el_c \end{bmatrix} = y_{ci} \text{ at } t_i$$

같은 시간 t_i 에서 관측을 하고 이 값으로 Non-linear function 방법을 이용한 Measurement model을 y_{ci} 로 나타낼 수 있다.

We use a first-order Taylor series expansion about a nominal trajectory.

$$y_{ci} = y_{ni} + \Delta r_i \frac{\partial y_{ni}}{\partial r_i} + \Delta r_j \frac{\partial y_{ni}}{\partial r_j} + \Delta r_k \frac{\partial y_{ni}}{\partial r_k} + \Delta v_i \frac{\partial y_{ni}}{\partial v_i} + \Delta v_j \frac{\partial y_{ni}}{\partial v_j} + \Delta v_k \frac{\partial y_{ni}}{\partial v_k}$$

where $\frac{\partial y_{ni}}{\partial r_i} = \frac{\partial y_{ci}}{\partial r_i} \Big|_{r_i, v_i}$ Taylor series를 통해 이 차이를 계속 줄여나간다.

The nominal trajectory is

$$y_{ni} = y(\vec{r}_n(t_i), \vec{v}_n(t_i))$$

The residuals $\vec{r}_i$ residual은 계산된 값과 실제 값의 차이를 말하며 다음과 같이 정의한다.

$$\begin{aligned} \vec{r}_i &= y_{0i} - y_{ci} \\ &= y_{0i} - \left[y_{ni} + \Delta r_i \frac{\partial y_{ni}}{\partial r_i} + \Delta r_j \frac{\partial y_{ni}}{\partial r_j} + \Delta r_k \frac{\partial y_{ni}}{\partial r_k} + \Delta v_i \frac{\partial y_{ni}}{\partial v_i} + \Delta v_j \frac{\partial y_{ni}}{\partial v_j} + \Delta v_k \frac{\partial y_{ni}}{\partial v_k} \right] \end{aligned}$$

Assume each measurement is weighted using its standard deviation

$$w_\rho = \frac{1}{\sigma_\rho}, \quad w_\beta = \frac{1}{\sigma_\beta}, \quad w_{el} = \frac{1}{\sigma_{el}} \quad \text{각 관측 값에 대해 가중치(w)를 부여한다.}$$

The cost function

$$J = \sum_{i=1}^N (w_i \bar{r}_i)^T (w_i \bar{r}_i) \quad \text{residual}(r) \text{을 통해 표현되는 Cos function}(J) \text{를 최소화 해야한다.}$$

$$\text{where } w_i = \begin{bmatrix} w_p & 0 & 0 \\ 0 & w_\beta & 0 \\ 0 & 0 & w_d \end{bmatrix}, \quad i=1,2,\dots,N$$

Set the first derivatives of J equals to zero.

$$\sum_{i=1}^N w_i^T w_i \bar{r}_i \frac{\partial \bar{r}_i}{\partial \vec{v}} = 0 \quad \text{이를 위해 미분한 값이 0이 되는 지점을 찾아야한다.}$$

$$\frac{\partial \bar{r}_i}{\partial \vec{v}} = \begin{bmatrix} \frac{\partial \bar{r}_i}{\partial v_1} \\ \frac{\partial \bar{r}_i}{\partial v_j} \\ \frac{\partial \bar{r}_i}{\partial v_k} \end{bmatrix}, \quad \sum_{i=1}^N \frac{\partial \bar{r}_i}{\partial \vec{v}} \Rightarrow \begin{bmatrix} \frac{\partial \bar{r}_1}{\partial v_1} & \frac{\partial \bar{r}_2}{\partial v_1} & \dots & \frac{\partial \bar{r}_N}{\partial v_1} \\ \frac{\partial \bar{r}_1}{\partial v_j} & \frac{\partial \bar{r}_2}{\partial v_j} & \dots & \frac{\partial \bar{r}_N}{\partial v_j} \\ \frac{\partial \bar{r}_1}{\partial v_k} & \frac{\partial \bar{r}_2}{\partial v_k} & \dots & \frac{\partial \bar{r}_N}{\partial v_k} \end{bmatrix} \approx - \begin{bmatrix} \frac{\partial y_{c1}}{\partial v_1} & \frac{\partial y_{c2}}{\partial v_1} & \dots & \frac{\partial y_{cN}}{\partial v_1} \\ \frac{\partial y_{c1}}{\partial v_j} & \frac{\partial y_{c2}}{\partial v_j} & \dots & \frac{\partial y_{cN}}{\partial v_j} \\ \frac{\partial y_{c1}}{\partial v_k} & \frac{\partial y_{c2}}{\partial v_k} & \dots & \frac{\partial y_{cN}}{\partial v_k} \end{bmatrix}$$

$$\therefore -1 \begin{bmatrix} \frac{\partial y_{c1}}{\partial r_1} & \frac{\partial y_{c2}}{\partial r_1} & \dots & \frac{\partial y_{cN}}{\partial r_1} \\ \frac{\partial y_{c1}}{\partial r_j} & \ddots & & \vdots \\ \vdots & & \ddots & \frac{\partial y_{cN}}{\partial v_j} \\ \frac{\partial y_{c1}}{\partial v_k} & \dots & \frac{\partial y_{cN-1}}{\partial v_k} & \frac{\partial y_{cN}}{\partial v_k} \end{bmatrix} \begin{bmatrix} w_1^2 \bar{r}_1 \\ w_2^2 \bar{r}_2 \\ \vdots \\ w_N^2 \bar{r}_N \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \text{위의 식을 정리한 결과이다}$$

$$\Leftrightarrow -A^T W \begin{bmatrix} \bar{r}_1 \\ \bar{r}_2 \\ \vdots \\ \bar{r}_N \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\text{where } W = \begin{bmatrix} w_1^2 & 0 & \dots & 0 \\ 0 & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \dots & 0 & w_N^2 \end{bmatrix}$$

위의 식을 두 항으로 분리하여 계산한다.

$$-A^T W \begin{bmatrix} y_{01} - y_{n1} \\ y_{02} - y_{n2} \\ \vdots \\ y_{0N} - y_{nN} \end{bmatrix} - A^T W \begin{bmatrix} \frac{\partial y_{n1}}{\partial r_1} & \frac{\partial y_{n1}}{\partial r_j} & \dots & \frac{\partial y_{n1}}{\partial v_k} \\ \frac{\partial y_{n2}}{\partial r_1} & \ddots & & \vdots \\ \vdots & & \ddots & \frac{\partial y_{nN-1}}{\partial v_k} \\ \frac{\partial y_{nN}}{\partial r_1} & \dots & \frac{\partial y_{nN}}{\partial v_j} & \frac{\partial y_{nN}}{\partial v_k} \end{bmatrix} \begin{bmatrix} \Delta r_1 \\ \Delta r_j \\ \Delta r_k \\ \Delta v_1 \\ \Delta v_j \\ \Delta v_k \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$A^T W \tilde{b} - A^T W A \delta \hat{X} = 0$$

$$\therefore \delta \hat{X} = (A^T W A)^{-1} A^T W \tilde{b} \quad \text{이를 간단히 나타낸 식이다.}$$

$$X_{new} = X_{old} + \delta \hat{X}$$

$$where\ X = \begin{Bmatrix} \vec{r}(t_0) \\ \vec{v}(t_0) \end{Bmatrix} \text{ at epoch time}$$

따라서 위의 식에서 RMS 값이 사용자가 설정한 범위 이하가 되면 epoch time에 수렴한다고 할 수 있으며 임의의 시간에 대한 위치와 속도를 구할 수 있다.

